Информационные технологии и программирование

3 семестр
Лекция 8.

Содержание лекции:

- Эволюция десктоп-разработки на C#
- WinForms
- WPF, XML, XSD
- WPF и MVVM

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Типы проектов под Desktop

Windows Forms

Windows Forms — интерфейс программирования приложений (API), отвечающий за графический интерфейс пользователя и являющийся частью Microsoft .NET Framework. Данный интерфейс упрощает доступ к элементам интерфейса Microsoft Windows за счет создания обёртки для существующего Win32 API в управляемом коде. Причём управляемый код — классы, реализующие API для Windows Forms, не зависят от языка разработки. То есть программист одинаково может использовать Windows Forms как при написании ПО на C#, C++, так и на VB.Net, J# и др.

Год рождения: ~2002, вместе с .NET Framework.

Философия: "Drag-and-Drop". Визуальный конструктор, быстрая прототипизация.

- **Плюсы:**
  - Очень низкий порог входа.
  - Быстрое создание простых приложений для Windows.
  - Огромная база знаний и готовых решений.
- **Минусы:**
  - Только для Windows.
  - Внешний вид привязан к версии Windows. Сложно кастомизировать.
  - Устаревшая архитектура (часто приводит к "спагетти-коду" с бизнес-логикой в обработчиках кнопок).

Windows Presentation Foundation (WPF)

Windows Presentation Foundation (**WPF**) — аналог WinForms, система для построения клиентских приложений Windows с визуально привлекательными возможностями взаимодействия с пользователем, графическая подсистема в составе .NET Framework, использующая язык разметки XAML[1].

WPF предустановлена в Windows Vista, Windows 7, Windows 8, Windows 8.1 и Windows 10.

Год рождения: 2006 (.NET Framework 3.0).

Философия: "Разделение ответственности". Внешний вид (XAML) отделен от логики (C#).

В основе WPF лежит векторная система визуализации, не зависящая от разрешения устройства вывода и созданная с учётом возможностей современного графического оборудования. WPF предоставляет средства для создания визуального интерфейса, включая язык XAML (eXtensible Application Markup Language), элементы управления, привязку данных, макеты, двухмерную и трёхмерную графику, анимацию, стили, шаблоны, документы, текст, мультимедиа и оформление.

Графической технологией, лежащей в основе WPF, является DirectX, в отличие от Windows Forms, где используется GDI/GDI+. Производительность WPF выше, чем у GDI+ за счёт использования аппаратного ускорения графики через DirectX.

Также существует урезанная версия CLR, называющаяся WPF/E, она же известна как Silverlight.

XAML представляет собой язык декларативного описания интерфейса, основанный на XML. Также реализована модель разделения кода и дизайна, позволяющая кооперироваться программисту и дизайнеру. Кроме того, есть встроенная поддержка стилей элементов, а сами элементы легко разделить на элементы управления второго уровня, которые, в свою очередь, разделяются до уровня векторных фигур и свойств/действий. Это позволяет легко задать стиль для любого элемента, например, Button (кнопка).

```
<StackPanel Orientation="Horizontal">
   <Slider x:Name="slider" Width="200" Minimum="1" Maximum="100"
Value="60"/>
   <TextBox Text="{Binding ElementName=slider, Path=Value,
Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}"/>
</StackPanel>
```

XML

Опубликован в 1998 г.

XML (eXtensible Markup Language) — это расширяемый язык разметки, предназначенный для хранения и передачи данных в структурированном, человекочитаемом текстовом формате.

Язык называется расширяемым, поскольку он не фиксирует разметку, используемую в документах: разработчик волен создать разметку в соответствии с потребностями к конкретной области, будучи ограниченным лишь синтаксическими правилами языка.

С физической точки зрения документ состоит из *сущностей* (англ. *entities*), из которых каждая может ссылаться на другую сущность. Единственный ***корневой элемент*** — **документная сущность**. Содержание сущностей — символы.

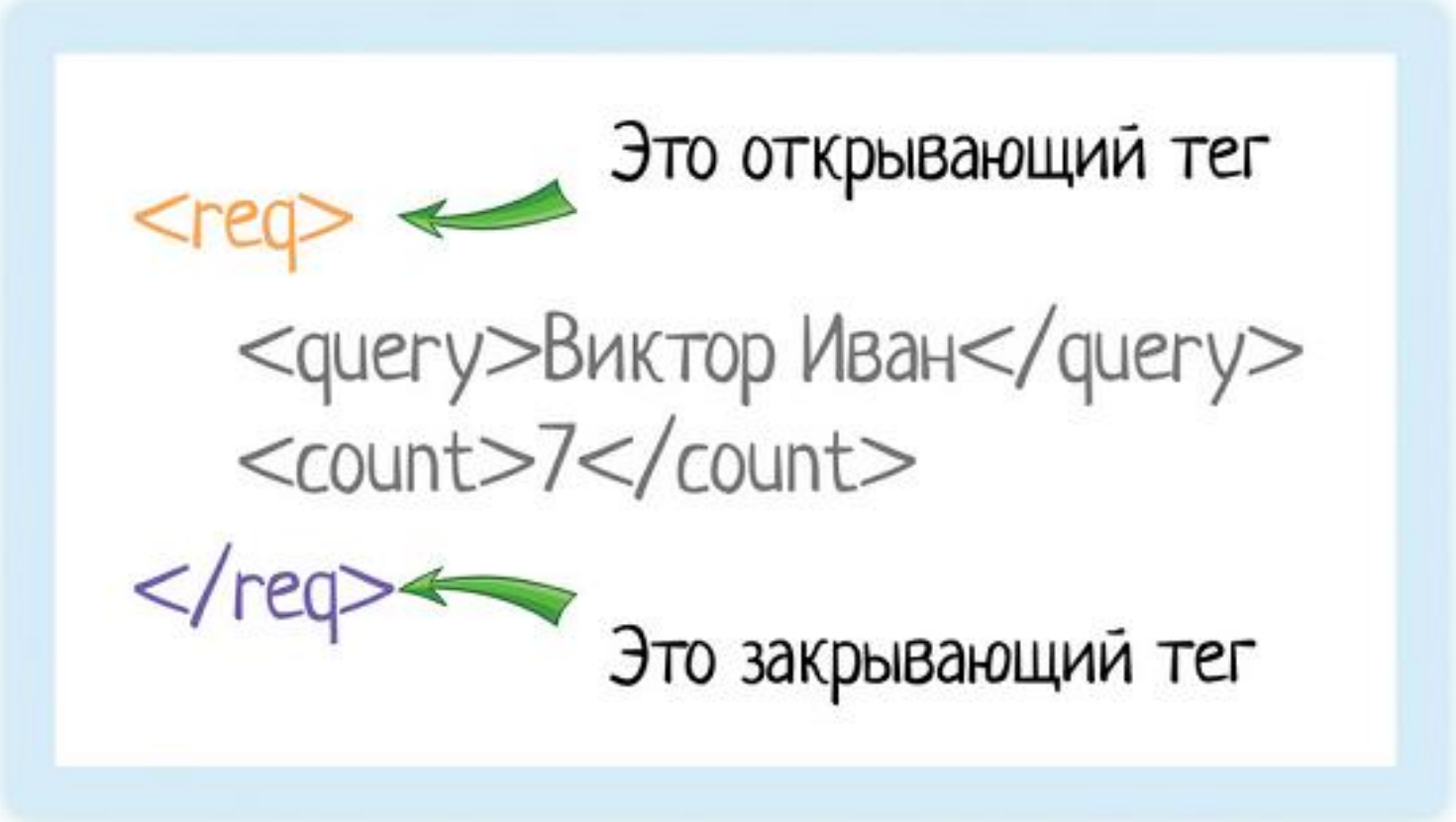
С логической точки зрения документ состоит из *комментариев* (англ. *comments*), *объявлений* (англ. *declarations*), *элементов* (англ. *elements*), *ссылок* на сущности (англ. *character references*) и *инструкций обработки* (англ. *processing instructions*). Всё это в документе структуризуется *разметкой* (англ. *markup*).

Сущность — мельчайшая часть в документе. Все сущности что-нибудь содержат, и у всех них есть имя (существуют исключения, напр. **документная сущность**).

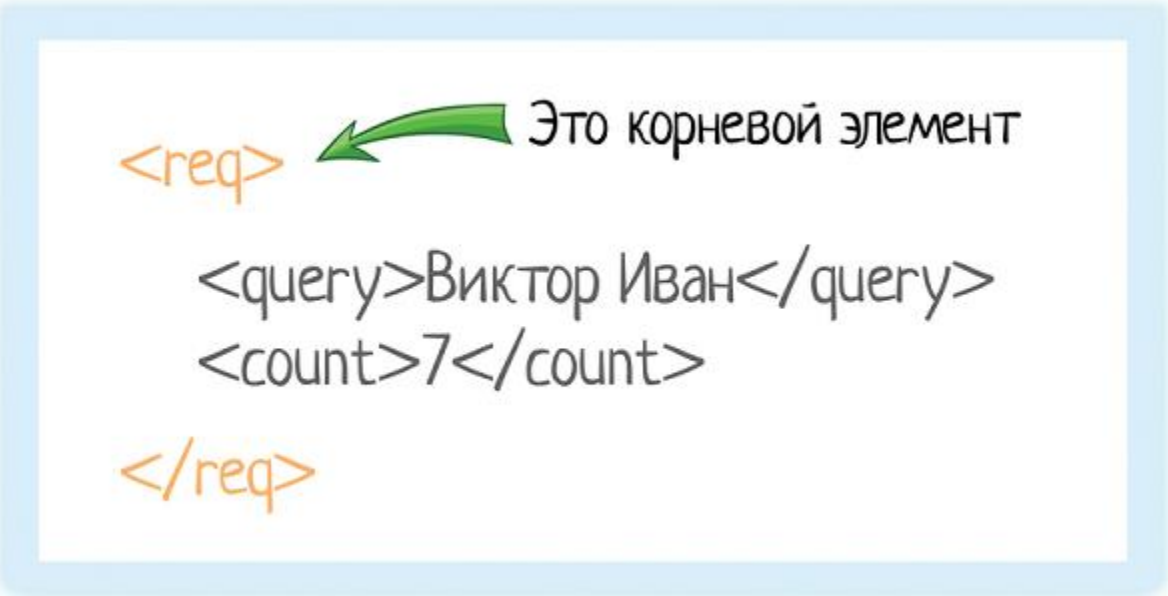
Документ состоит из сущностей, содержание которых — символы. Все символы разделены на два типа: символы данных и символы разметки. К разметке относятся:

1. теги (англ. *tags*) - обозначают границы элементов;
2. объявления и инструкции обработки, включая их *атрибуты* (англ. *attributes*);
3. ссылки на сущности;
4. комментарии;
5. последовательности символов, обрамляющие секции «CDATA».

В XML каждый элемент должен быть заключен в теги. **Тег** — это некий текст, обернутый в угловые скобки:



Все составляющие части документа обобщаются в *пролог* и **корневой элемент**. **Корневой элемент** — обязательная часть документа, составляющая всю его суть (пролог, вообще говоря, может отсутствовать). Корневой элемент может включать (а может не включать) вложенные в него элементы, символьные данные и комментарии. Вложенные в корневой элемент элементы, в свою очередь, могут включать вложенные в них элементы, символьные данные и комментарии, и так далее.



Пролог может включать **объявления**, **инструкции обработки**, **комментарии**. Его следует начинать с *объявления XML*, хотя в определённой ситуации допускается отсутствие этого объявления.

```
<?xml version="1.0" encoding="UTF-8"?>
<contactList>
    <contact>
        <firstName>Иван</firstName>
        <lastName>Иванов</lastName>
        <phone>+71234567890</phone>
        <email>ivanov@example.com</email>
    </contact>
    <contact>
        <firstName>Анна</firstName>
        <lastName>Петрова</lastName>
        <phone>+70987654321</phone>
        <email>petrova@example.com</email>
    </contact>
</contactList>
```

Элементы документа должны быть *правильно вложены*: любой элемент, начинающийся внутри другого элемента (то есть любой элемент документа, кроме корневого), должен заканчиваться внутри элемента, в котором он начался. Символьные данные могут встречаться внутри элементов как непосредственно так и в специальных *секциях «CDATA»*. Объявления, инструкции обработки и элементы могут иметь связанные с ними атрибуты. Атрибуты используются для связывания с логической единицей текста пар «имя-значение».

Атрибуты. Это дополнительные данные, которые можно добавить к элементам, чтобы более точно указать их характеристики. Они записываются внутри открывающего тега и позволяют добавлять дополнительную информацию без создания новых элементов. Например, в элементе **<book genre="учебник">** атрибут **genre** указывает на жанр книги, что помогает более точно описать её содержание.

Атрибуты элемента party

```
<party type="PHYSICAL" sourceSystem="AL" rawId="2">
    <field name="name">Олег </field>
    <field name="birthdate">02.01.1980</field>
    <attribute type="PHONE" rawId="AL.2.PH.1">
        <field name="type">MOBILE</field>
        <field name="number">+7 916 9754221</field>
    </attribute>
</party>
```

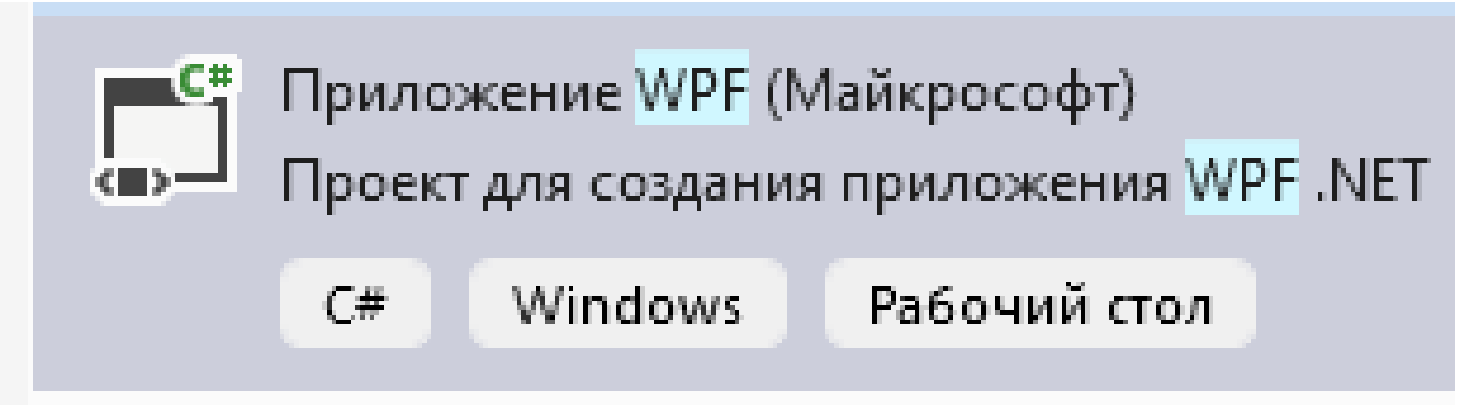
XSD

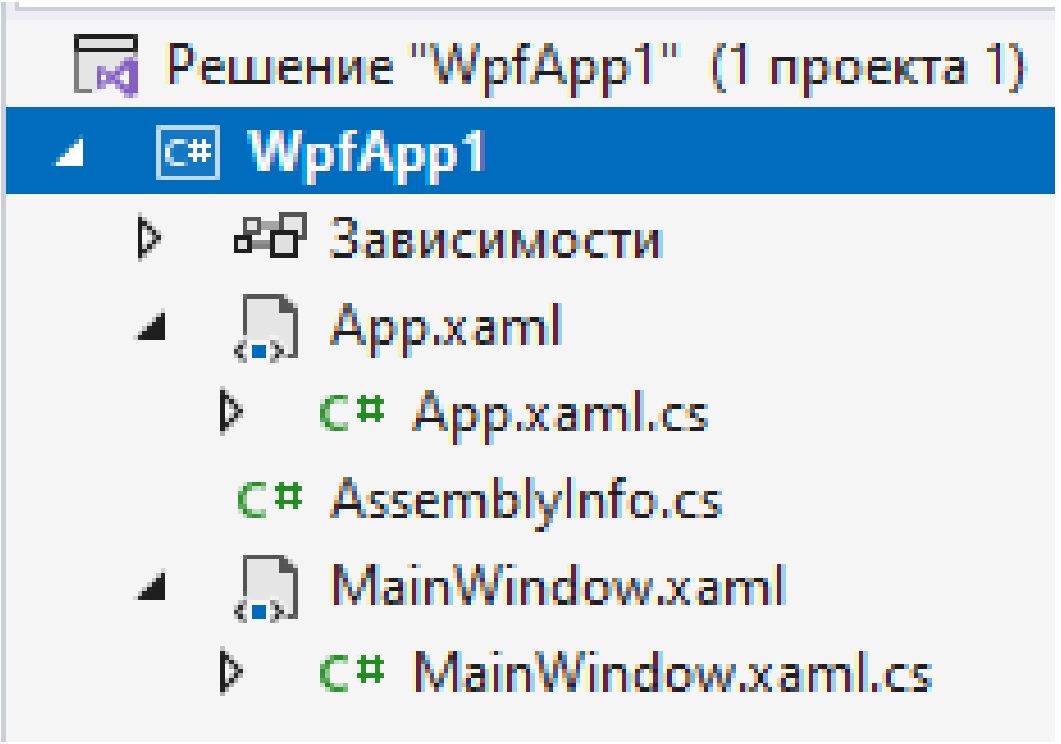
Опубликован в 2001 г.

XSD (**X**ML **S**chema **D**efinition) — это описание XML – как он должен выглядеть, что в нем должно быть.

```
<xs:element name="doRegister ">
   <xs:complexType>
      <xs:sequence>
        <xs:element name="email" type="xs:string"/>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="password" type="xs:string"/>
      </xs:sequence>
   </xs:complexType>
</xs:element>
```

Программа на C# WPF





Проект состоит из:

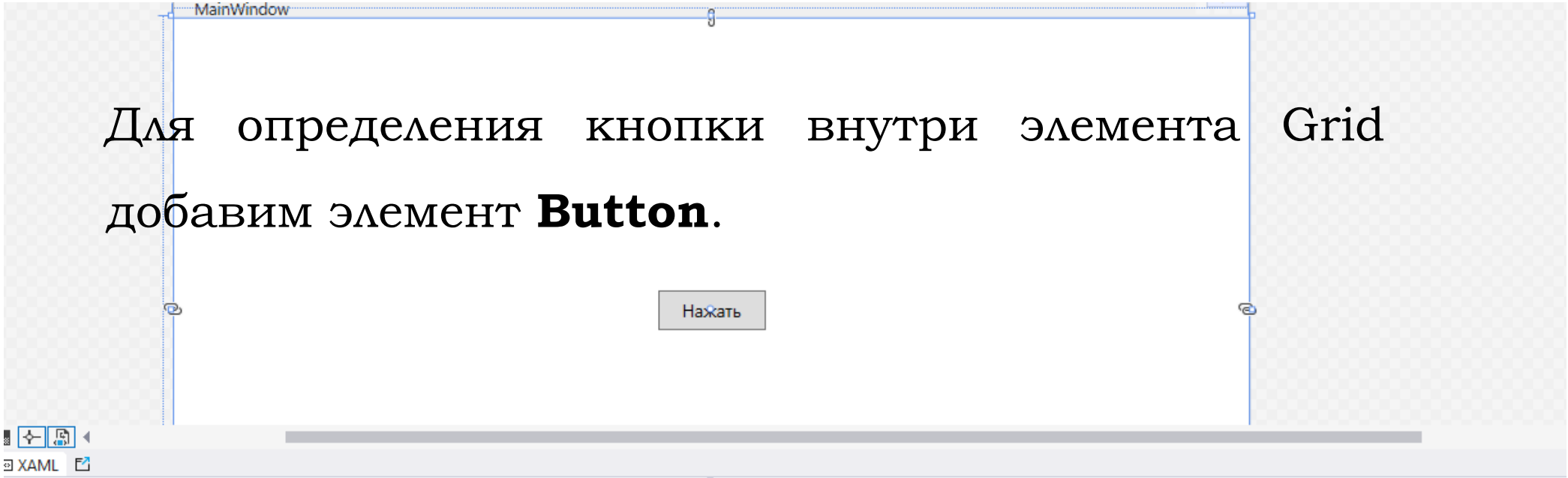
1. Файла окна приложения – **MainWindow.xaml**
2. Файл логики кода окна приложения - **MainWindow.xaml.cs**.

Файла окна приложения **MainWindow.xaml**

Здесь у нас сначала определен элемент верхнего уровня Window - окно приложения, в нем определен элемент Grid - контейнер верхнего уровня, в который мы можем добавлять другие элементы.

```
<Window x:Class="WpfApp1.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfApp1"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
    <Grid>

    </Grid>
</Window>
```

Для определения кнопки внутри элемента Grid добавим элемент **Button**.



Button

```
<Window x:Class="WpfApp1.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfApp1"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
    <Grid>
        <Button Content="Нажать" FontSize="13" Width="80" Height="30"
                Click="Button_Click" />
    </Grid>
</Window>
```

Здесь добавлен метод Button_Click, который будет выступать в качестве обработчика нажатия кнопки. Ообработчики нажатия кнопки должны принимать два параметра типов object и RoutedEventArgs. В самом обработчике вызывается метод MessageBox.Show, который отображает окно с сообщением. Отображаемое сообщение передается в качестве параметра.

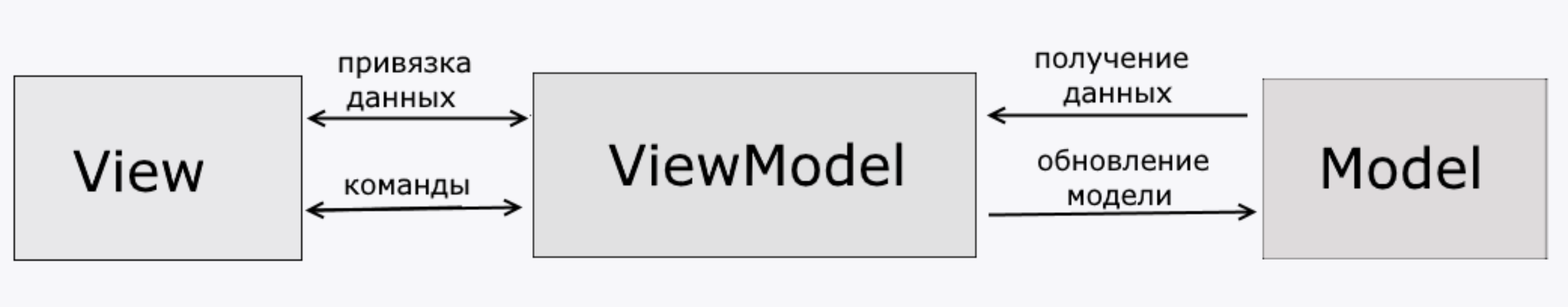
```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show("WPF на METANIT.COM");
    }
}
```

WPF и MVVM

Паттерн **MVVM (Model-View-ViewModel)** позволяет отделить логику приложения от визуальной части (представления). Данный паттерн является архитектурным, то есть он задает общую архитектуру приложения.

Данный паттерн был представлен Джоном Госсманом (John Gossman) в 2005 году как модификация шаблона Presentation Model и был первоначально нацелен на разработку приложений в WPF. И хотя сейчас данный паттерн вышел за пределы WPF и применяется в самых различных технологиях, в том числе при разработке под Android, iOS, тем не менее WPF является довольно показательной технологией, которая раскрывает возможности данного паттерна.

MVVM состоит из трех компонентов: модели (Model), модели представления (ViewModel) и представления (View).



Модель описывает используемые в приложении данные. Модели могут содержать логику, непосредственно связанную этими данными, например, логику валидации свойств модели. В то же время модель не должна содержать никакой логики, связанной с отображением данных и взаимодействием с визуальными элементами управления.

Нередко модель реализует интерфейсы **INotifyPropertyChanged** или **INotifyCollectionChanged**, которые позволяют уведомлять систему об изменениях свойств модели. Благодаря этому облегчается привязка к представлению, хотя опять же прямое взаимодействие между моделью и представлением отсутствует.

View или представление определяет визуальный интерфейс, через который пользователь взаимодействует с приложением. Применительно к WPF представление - это код в xaml, который определяет интерфейс в виде кнопок, текстовых полей и прочих визуальных элементов.

Хотя окно (класс Window) в WPF может содержать как интерфейс в xaml, так и привязанный к нему код C#, однако в идеале код C# не должен содержать какой-то логики, кроме разве что конструктора, который вызывает метод InitializeComponent и выполняет начальную инициализацию окна. Вся же основная логика приложения выносится в компонент ViewModel.

не обрабатывает события за редким исключением, а выполняет действия в основном посредством команд.

ViewModel или модель представления связывает модель и представление через механизм привязки данных. Если в модели изменяются значения свойств, при реализации моделью интерфейса INotifyPropertyChanged автоматически идет изменение отображаемых данных в представлении, хотя напрямую модель и представление не связаны.

ViewModel также содержит логику по получению данных из модели, которые потом передаются в представление. И также VewModel определяет логику по обновлению данных в модели.

Поскольку элементы представления, то есть визуальные компоненты типа кнопок, не используют события, то представление взаимодействует с ViewModel посредством команд.

Например, пользователь хочет сохранить введенные в текстовое поле данные. Он нажимает на кнопку и тем самым отправляет команду во ViewModel. А ViewModel уже получает переданные данные и в соответствии с ними обновляет модель.

Итогом применения паттерна MVVM является функциональное разделение приложения на три компонента, которые проще разрабатывать и тестировать, а также в дальнейшем модифицировать и поддерживать.

Программа на WPF + MVVM

1. MainViewModel.cs (вся логика)

```
using System.ComponentModel;
using System.Windows.Input;

namespace WpfApp1;

public class MainViewModel : INotifyPropertyChanged
{
// Содержимое далее
}
```

```
private string _text = "Привет!";
private int _count = 0;

public string Text
{
   get => _text;
   set
   {
      _text = value;
      OnPropertyChanged(nameof(Text));
   }
}
```

```
public int Count
{
   get => _count;
   set
   {
      _count = value;
      OnPropertyChanged(nameof(Count));
      OnPropertyChanged(nameof(DisplayText));
      // Автоматическое обновление!
   }
}
public string DisplayText => $"Нажато: {Count} раз";
public ICommand ClickCommand => new RelayCommand(() => Count++);
public event PropertyChangedEventHandler? PropertyChanged;
protected void OnPropertyChanged(string name) =>
   PropertyChanged?.Invoke(this,
                           new PropertyChangedEventArgs(name));
```

Вспомогательный класс:

```
using System.Windows.Input;

namespace WpfApp1;

public class RelayCommand : ICommand
{
    private Action _execute;
    public RelayCommand(Action execute) => _execute = execute;
    public bool CanExecute(object parameter) => true;
    public void Execute(object parameter) => _execute();
    public event EventHandler CanExecuteChanged;
}
```

2. MainWindow.xaml (XAML) - красивый интерфейс

Вспомогательные настройки:

```
<Window
   x:Class="WpfApp1.MainWindow"
   xmlns=http://schemas.microsoft.com/winfx/2006/xaml/presentation
   xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
   xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
   xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
   xmlns:local="clr-namespace:WpfApp1"
   mc:Ignorable="d"
   Title="WPF MVVM" Height="200" Width="300">
   <!-- Содержимое далее -->
</Window>
```

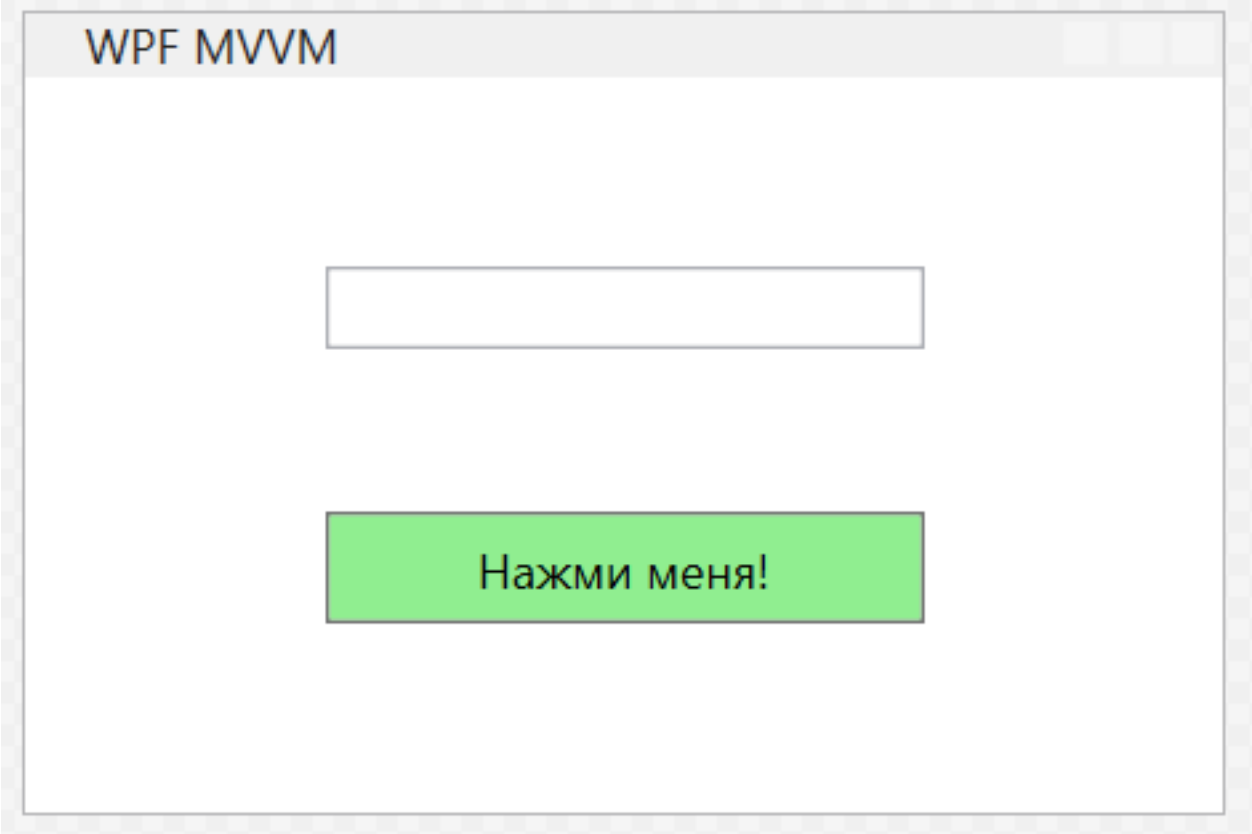
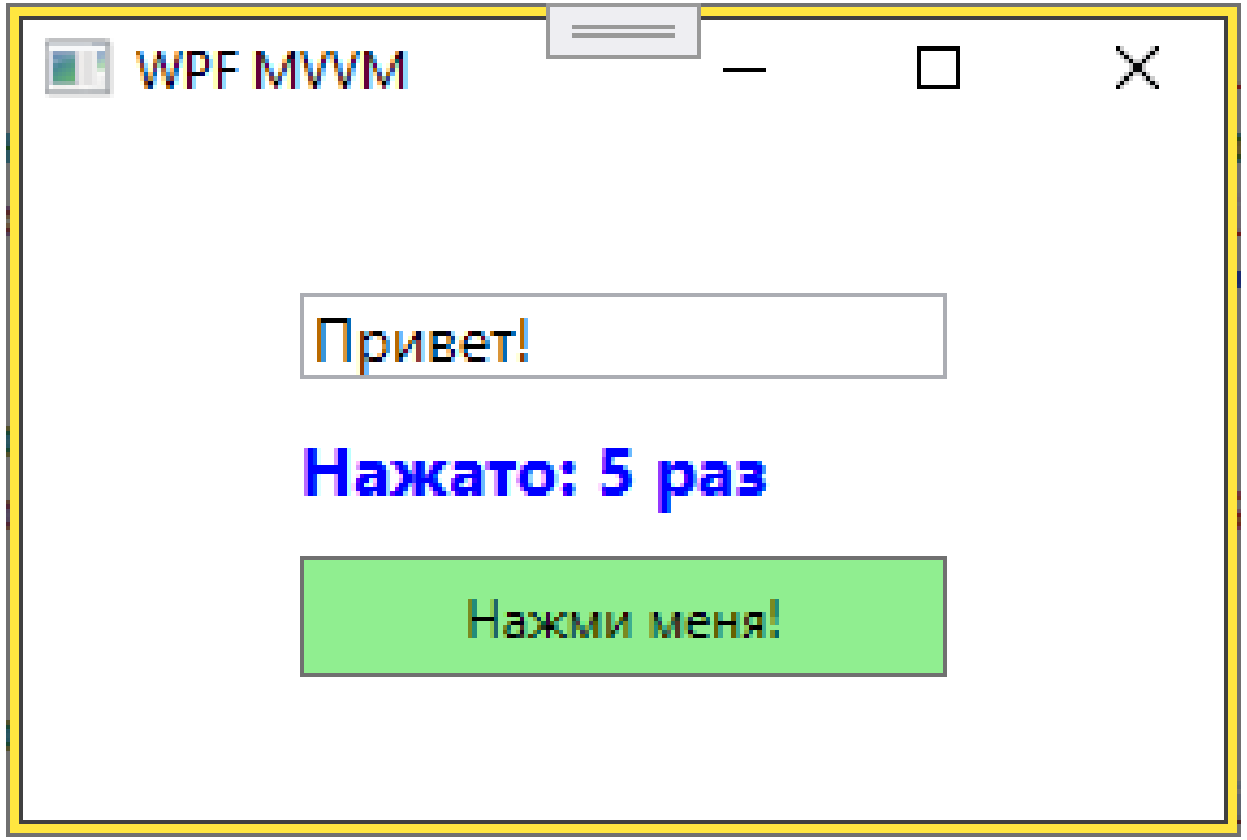
Важно чтобы совпадало название класса (MainWindow)

```
<StackPanel
   VerticalAlignment="Center" HorizontalAlignment="Center">
   <!-- Текстовое поле СРАЗУ связано со свойством Text -->
   <TextBox Text="{Binding Text,
                           UpdateSourceTrigger=PropertyChanged}"
   FontSize="14" Width="150" Margin="5"/>

   <!-- Отображаем вычисляемое свойство -->
   <TextBlock Text="{Binding DisplayText}"
              FontSize="16" FontWeight="Bold" Foreground="Blue"
              Margin="5"/>

   <!-- Кнопка вызывает команду -->
   <Button Content="Нажми меня!"
           Command="{Binding ClickCommand}"
           Background="LightGreen" Padding="10,5" Margin="5"/>
</StackPanel>
```

Осталось только привязать контекст данных:

```
using System.Windows;
namespace WpfApp1;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new MainViewModel();
    }
}
```

В дизайнере:



Результат: